

[L0-L1 核心本质与设计逻辑]

L0 一句话本质：在非易失性随机读写受限的介质上，通过内存无锁跳表配合顺序追加日志，以及后台多线程级联归并压实，将零散的随机写转化为顺序写，构建极致吞吐与快强隔离的嵌入式键值存储引擎。

L1 四句话逻辑：

- 内存无锁跳表与追加写** (M1 Write) 依靠顺序追加 WAL 保障可靠性，利用 CAS 乐观并发与 Arena 内存池消除锁竞争与碎片。
- SSTable 紧凑分块与前缀检索** (M2 SSTable) 通过重启点前缀共享降低物理存储开销，配合 Bloom Filter 在磁盘检索前快速排除无效 I/O。
- 分层压实与三大放大折中** (M3 Compaction) 借由后台级联归并回收死空间并维持全局键有序，在写放大、读放大与空间放大之间实施动态博弈。
- 无锁并发批处理与 Sequence 快照** (M5 Transaction) 将复杂逻辑剥离，通过全局序列号构建轻量 MVCC，支持 WriteBatch 原子并轨写与 2PC 协调。

[L2 架构演进拓补]

• [用户写 WriteBatch] → [顺序追加 WAL] → [并发写入 MemTable] → [写满冻结 Immutable MemTable] → [后台单线程 Flush] → [L0 SSTable 磁盘文件] → [后台多线程 Leveled Compaction 合并] → [Block Cache 块读取缓存] → [Row Cache 行高速缓存] → [Snapshot 强一致性快照读]

[M1.1] 写入链路 with WAL 日志 (Write Flow & WAL)

- 物理链路：**用户写请求 (Put/Delete) 到达后，首先以追加方式顺序写入磁盘的预写日志 (WAL)。随后该修改被同步应用到内存中的 **MemTable**。一旦完成这两步，写操作即宣告成功。
- 同步参数：**通过 WriteOptions.sync 参数控制落盘时机。若设为 true，每次写入强制调用 fsync 确保数据在磁盘盘片或固态硬盘上持久化，防止突然掉电导致 Page Cache 脏数据丢失。

[M1.2] MemTable 无锁跳表机制 (SkipList MemTable)

- 跳表结构：**默认的 MemTable 实现是基于内存的并发无锁跳表 (SkipList)。利用指针的多层索引结构，实现 $O(\log N)$ 的极速定位与范围扫描。
- CAS 并发：**跳表层级指针修改使用基于 Compare-And-Swap (CAS) 的乐观无锁并发控制，多线程写入无需加全局排他锁，配合 Arena 分配器预先从操作系统申请大连续物理内存，自行管理行级空间小分配，有效规避了标准 malloc 带来的严重内存碎片。

[M1.3] MemTable 物理分类选型 (MemTable Types)

- 分类选型：**
 - SkipListMemTab** (默认)：通用型，支持高并发读写及高效的区间扫描；
 - VectorMemTab**：底层为 C++ std::vector，无并发写入能力。仅适合大批量离线数据一次性单线程载入，其检索依靠全表排序后二分；
 - HashSkipList / HashLinkList**：在跳表之上加了一层 Hash 桶。针对有固定前缀 (Prefix) 的键，可极大加快速度查询 (Point Lookup) 并支持前缀限定扫描。

[M1.4] Immutable MemTable 与 Flush 刷盘 (Immutable MemTable & Flush)

- 状态冻结：**当活跃的 MemTable 写入量达到 write_buffer_size 限时，它会被标记为只读的 **Immutable MemTable** (不可变内存表)，同时分配一个新的活跃 MemTable 继续接收写流量。
- Flush 链路：**后台单线程 Flush 调度器被唤醒，将 Immutable MemTable 中的数据按顺序持久化输出到磁盘的 L0 层 (Level 0) SSTable 文件中。刷盘完成后，对应的 WAL 空间被标记回收。

[M1.5] Write Stall 写入限速挂起机制 (Write Stall)

- 限速机制：**LSM 引擎的核心自卫系统。当后台 Flush 或 Compaction 线程速度赶不上客户端写入速度时，为防内存爆满或磁盘完全跑挂，RocksDB 会主动减缓甚至暂停客户端写入。
- 触发红线：**
 - L0 层 SSTable 文件数达到 level0_slowdown_writes_trigger (默认 20，开始限制写吞吐)；
 - 达到 level0_stop_writes_trigger (默认 30，写完全挂起冻结)；
 - 待压实字节数 (Pending Compaction Bytes) 超限。

[M2.1] SSTable 整体物理架构 (SSTable Block-Based Table)

- 物理布局：**磁盘上的 SSTable 采用块结构 (Block-based Table Format)。数据按顺序划分为多个固定大小的 **Data Block** (默认 4KB)。文件末尾包含 **Filter Block** (布隆过滤器)、**Index Block** (索引块)、**MetaIndex Block** 及 48 字节固定大小的 **Footer** (尾锚定区)。
- 读取路径：**读取时，首先读取 Footer，Footer 指向 MetaIndex Block，再由 MetaIndex 指向 Index 和 Filter 块，进而精确定位数据块，防止全表扫描。

[M2.2] Block 内部结构与前缀压缩 (Block Format & Restart Points)

- 前缀共享：**Block 内部的键是严格排序的。为节省空间，后续键只存储与前一个键不相同的后缀，以 shared_bytes/unshared_bytes/value 的形式进行前缀共享存储。
- 重启点二分：**为了能在 Block 内进行随机检索，每隔固定个键 (例如 16 个) 就会中断前缀共享，写入一个完整的键作为 **Restart Point** (重启点)。Block 末尾存有所有重启点的物理偏移数组。检索时，对重启点数组进行折半二分定位，再在 16 个键内顺序扫描。

[M2.3] 布隆过滤器 (Bloom Filter) 布局 (Bloom Filters)

- 排重过滤：**布隆过滤器利用多个哈希函数将键映射到位图数组中。通过 $O(1)$ 的位运算，在访问磁盘 Block 前判定“该键是否绝对不存在”。
- 布局版本：**
 - Block-based Filter** (旧版)：为每个 2KB 的数据块独立生成布隆过滤器，缺点是无法对全文件进行一次性过滤定位；
 - Full Filter** (新版，生产首选)：为整个 SSTable 文件只生成一个连续的全局过滤器，对文件级点查询可减少一次物理 I/O。

[M2.4] Index Block 与二分检索 (Index Block)

- 索引边界：**Index Block 存放了指向各个 Data Block 的索引项。每个索引项的 Key 是对应 Data Block 中最大 Key 的一个紧凑“前缀分割线” (Shortest Separator Key)，Value 是该 Data Block 在文件中的物理偏移与长度。
- 哈希二分：**支持 Binary Search (标准二分查找索引) 与 Hash Search (前缀哈希二分，利用前缀 Bloom Filter 先行过滤，消除在 Index 块中盲目二分的时间)。

[M3.1] Size-Tiered Compaction 机制 (Size-Tiered Compaction)

- 机制原理：**将大小相似的 SSTable 文件归为一组 (Tier)。当该组文件数达到设定阈值时，后台线程将它们读入并进行多路归并排序，合并输出为一个更大体积的 SSTable 移入下一组。
- 放大折中：**写入放大 (Write Amplification) 最低 (只需顺序合并)；但空间放大 (Space Amplification) 极高，且在合并大文件时，需要至少一倍于数据的磁盘空闲临时空间，易引发“磁盘满死锁”。

[M3.2] Leveled Compaction 机制 (Leveled Compaction)

- 分层设计：**磁盘空间严格划分为 Level 0 至 Level Max。L0 的 SSTable 文件间键范围允许重叠，其它各层 (L1-LMax) 文件内部严格有序且文件间键范围绝不重叠。每层总大小设定为 10 倍级联递增 (如 L1=256MB, L2=2.5GB)。
- 合并流转：**当 L_n 层总字节数超限时，选择该层一个 SSTable，与 L_{n+1} 层中与其键范围重叠的所有 SSTable 进行多路归并排序合并，在每一层消除脏数据和墓碑标记。空间利用紧凑，空间放大极低；但写放大非常高 (通常为 10-30)。

[M3.3] Compaction Pick 压实选择策略 (Compaction Scores)

- 计分公式：**RocksDB 后台线程通过计算每一层的 **Compaction Score** 决定对哪一层进行压实合并：
 - 对于 L0，Score = 当前 L0 文件数 / level0_file_num_compaction_trigger；
 - 对于 $L_n (n \geq 1)$ ，Score = 当前层实际大小 / 该层最大容量限制。
- 调度执行：**挑选 Score 最大的层级。并在该层级内优先选择生存时间长、或与下一层重叠度最小的 SSTable 执行合并，以最小化 I/O 开销。

[M3.4] Write/Space/Read Amplification 放大定理 (Amplification Law)

- 读写空间折中：**LSM-Tree 引擎底层的物理宿命。
- 写放大 (WA)：**写入 1 字节数据导致实际写入磁盘的总字节数。WA 高会缩短 SSD 寿命并引发 stall；
- 空间放大 (SA)：**磁盘实际占用空间与有用数据逻辑体积的比值。SA 高导致物理容量开销大；
- 读放大 (RA)：**单次 Get 操作实际读取的物理字节与逻辑键值的比值。RA 高拖累读取 QPS，Leveled 用高 WA 换取了极佳的 SA 与稳定的 RA。

[M3.5] FIFO Compaction 与时序存储 (FIFO Compaction)

- 时序极速：**最纯粹的极速压实策略。SSTable 仅在 L0 层存在，所有新刷盘的文件按时间顺序排列。一旦所有文件总容量超过 max_table_size_definition，直接物理删除最老的 SSTable 文件。
- 零放大：**写放大严格等于 1，完全没有后台归并排序的磁盘 I/O 损耗。只适用于允许过期自动丢弃的时序监控、事件埋点及定长数据缓存场景。

[壹] RocksDB 经典架构折衷与异常防范矩阵 (RocksDB Architectural Trade-offs)

- 直覺 1：**既然 RocksDB 支持 WriteBatch，我们可以放心地将几十万条海量更新塞进一个 Batch 中进行原子一次性写入以提升速度。 **解构：**WriteBatch 必须在内存中完整解析并生成 MemTable 节点。超大 Batch 会强行挤爆内存引起严重的 STW (Stop-The-World) 内存再分配，且会长时间独占写领导致整个引擎其它线程饿死。 → 必须拆分超大 Batch。推荐单个 WriteBatch 包含的 Key 数量在 1,000 到 10,000 之间。大批量导入应采用多线程生成 SST File Writer 并直接 Ingest 导入。
- 直覺 2：**Block Cache 的大小设置得越充足，读吞吐量就越高；我们应当直接把所有物理内存全部分配给 Block Cache。 **解构：**LSM 引擎在读取时会并发搜索多个 L0 文件的 Index 和 Filter。如果把所有内存分给 Block Cache，会导致 MemTable 的写入 Arena 空间不足，频繁触发 Flush；且内核 Page Cache 缺失会导致物理磁盘读放大剧变。 → 严格控制 Block Cache 配比。一般为其分配物理内存的 30% 50%，并确保开启 cache_index_and_filter_blocks (将索引和过滤器也放入缓存管理)，设置 WriteBufferManager 控制全局 MemTable 内存总界。
- 直覺 3：**Leveled Compaction 性能极好且空间紧凑；我们在任何场景下都应该将 RocksDB 强制设定为 Leveled Compaction 模式。 **解构：**在持续大规模吞吐随机写入场景下，Leveled Compaction 层级间 10 倍的比率会导致严重的写放大 (WA 达 20-30 倍)，抢占物理 I/O 带宽，触发 Write Stall，严重拖累前台读写时延。 → 针对高写入时序数据 (如监控指标、归档日志)，必须选用 FIFO Compaction 或 Universal Compaction，以牺牲部分读取性能换取写放大降低至 1.5-2 左右。
- 直覺 4：**既然我们设置了 sync=false 以加速随机写，操作系统在掉电时依然能像 sync=true 一样保证 RocksDB 的事务绝对零丢失。 **解构：**sync=false 仅将 WAL 数据顺写到系统 Page Cache，并未真正落盘。机器一旦遭遇硬件断电，Page Cache 内尚未刷脏的脏数据将永久丢失，破坏分布式事务一致性。 → 针对强一致性场景，必须在 WriteOptions 中显式设置 sync=true。若为了吞吐折中，可采用组提交 (Group Commit) 或半同步落盘策略 (利用 RocksDB WriteThread 自动开机关合并写)。
- 直覺 5：**我们在多线程中并发使用 Get() 检索键，不需要配置任何 Block Cache 也能获得确定的微秒级检索响应。 **解构：**LSM 树没有全局就地索引。如果 Block Cache 缺失，每一次 Get() 必须遍历多层 SST 的二分索引块，甚至多次回表读取数据块，导致产生随机磁盘 I/O 穿透，检索时延暴增百倍。 → 必须配置 Block Cache 并设置 pin_l0_filter_and_index_blocks_in_cache=true。确保 L0 文件的索引与过滤器常驻物理内存，防止随机穿透。

[M4.1] Block Cache 块缓存架构 (Block Cache)

- **内存对象缓存**：用于缓存从磁盘 SSTable 解压读取出来的 Data Block 结构。默认提供 LRU Cache（最小最近未使用算法，底层为双向链表 + 哈希表）与 Clock Cache（环形指针扫描，并发度更高）。
- **锁分离设计**：为防止多线程并发读取缓存引发全局互斥锁的自旋等待瓶颈，Block Cache 采用 Sharding（分片）设计（默认分 16 片），通过哈希将 Key 分流到不同片区，大幅降低锁粒度。

[M4.2] Compressed Block Cache 选型 (Compressed Cache)

- **二级缓存**：位于 Block Cache 与物理磁盘之间的中转站。Compressed Block Cache 直接在内存中缓存从磁盘加载出来的“原始压缩字节块”（未解压）。
- **权衡机制**：
 - Uncompressed Cache（Block Cache）：直接存储 C++ 对象，省去 CPU 解压力力，但内存占用大；
 - Compressed Cache：内存占用小（能存更多块），但每次命中读取需要消耗 CPU 执行解压计算。仅当系统 CPU 算力富余而物理内存极度紧张时开启。

[M4.3] Row Cache 行缓存机制 (Row Cache)

- **行级高速路**：比 Block Cache 更上一層的缓存。它直接缓存逻辑键值对（Key-Value），而非物理磁盘块。
- **检查分流**：读取时，首先检索 Row Cache。若命中，可直接返回结果，彻底免除在 Block Cache 内进行 Block 索引二分查找的 CPU 计算开销。若未命中，则回到 Block Cache 进行标准 Block 查找，并在读取后将结果回写 Row Cache。

[M4.4] Index/Filter 内存 Pinning 锁定 (Pinning Index & Filters)

- **读穿防护**：由于 SSTable 的 Index 和 Filter 块体积很大，如果被 Block Cache 的 LRU 淘汰置换出内存，会导致下一次查询时发生 2-3 次随机磁盘 I/O 物理读取。
- **锁定常驻**：必须在 BlockBasedTableOptions 中配置 pin_l0_filter_and_index_blocks_in_cache=true，甚至开启 pin_top_level_index_and_filter，将 L0 文件以及其它各层 SSTable 的顶级元数据强行 Pin（锁死）在物理内存中，绝不参与 LRU 淘汰，捍卫读时延稳定。

[M5.1] ReadOptions Snapshot 快照机制 (Snapshots & MVCC)

- **版本视图**：RocksDB 内部对每次写操作都会递增全局唯一的 Sequence Number（序列号）。创建快照时，数据库分配当前的 Sequence Number 给该快照。
- **快照可见性**：之后的 Get/Iterator 读请求携带该 Snapshot，可见性判定规则为：只可读取到 SequenceNumber_record <= SequenceNumber_snapshot 的已提交记录。读请求完全不加任何锁，实现高性能快照读。

[M5.2] WriteBatch 原子写机制 (WriteBatch & Group Commit)

- **写入并轨**：用户并发写入时，写请求在内部被塞入一个全局的 WriteThread 队列。通过“并轨合并（Group Commit）”，队列的第一个线程被选为 Leader，其它线程为 Follower。
- **独占落盘**：Leader 线程获取排他写锁，将自己以及所有 Follower 的 Batch 串联起来，一次性顺序写入 WAL，并代表大家统一进行 Flush。随后，各个 Follower 线程并发地将自己 Batch 内的数据并行写入各自的 MemTable，在保障原子性的同时消除了线程调度切换开销。

[M5.3] Optimistic Transaction 乐观事务 (Optimistic Transactions)

- **冲突检测**：在执行 Put/Delete 时不加分行级锁，而是记录当前事务开始时的 Read Snapshot SequenceNumber。
- **提交回滚**：当事务尝试 Commit 时，在内部锁保护下，利用 Conflict Detection 机制检查这些 Key 从事务启动到提交之间是否产生过更高的 Sequence Number 写入。若有，代表发生了并发冲突，回滚整个 WriteBatch。适合写争用较低、冲突较少的场景。

[M5.4] Pessimistic Transaction 悲观事务 (Pessimistic Transactions)

- **行级加锁**：通过行锁管理器（LockManager）在写入或显式读取（Get/ForUpdate）时对 Key 加独占行级锁（Exclusive Lock）。
- **死锁图排查**：多线程并发等待行锁时，若在设定时延内未获取锁，会触发 Deadlock Detector 运行。它在内存中构建“等待持有图”（Wait-For Graph），一旦检测到有向环路（死锁），强制回滚其中某一个代价最小的事务以释放锁。

[M5.5] Two-Phase Commit (2PC) 分布式事务 (Two-Phase Commit)

- **分布式协调**：当 RocksDB 作为外部分布式引擎（如 TiDB / CockroachDB）的底层存储时，支持两阶段提交（2PC）。
- **Prepare 落盘**：第一阶段，将事务的 Prepare 状态及数据写入 WAL 并生成特定的 Prepare Marker，此时数据不可见；第二阶段，待分布式协调器发送 Commit 指令后，写入 Commit Marker，数据正式对其它 Snapshot 可见。

[M6.1] Column Families 列族隔离 (Column Families)

- **逻辑分库**：将一个物理 RocksDB 实例划分为多个独立的逻辑分区（列族，如 Metadata 列族、Data 列族）。列族之间共享同一个全局 WAL 顺序日志以保证崩溃恢复原子性。
- **独立控制**：每个列族拥有自己独立的 MemTable、SSTable 磁盘文件、Block Cache 缓存以及 Compaction 压实合并调度策略，实现多业务在单实例内的资源按需精细化隔离。

[M6.2] SST File Writer 离线批量导入 (SSTFileWriter)

- **旁路提速**：针对千万级甚至亿级数据的大规模批量数据加载（Bulk Loading），绕过 RocksDB 昂贵的前台写入链（WAL、MemTable 锁竞争、并发冲突及后台 Flush）。
- **直接写入**：在应用内使用 SSTFileWriter 在内存直接生成完全排序的 SSTable 物理文件，然后调用 IngestExternalFile 接口将文件硬链接或移动到 RocksDB 磁盘目录下。引擎只需修改 MANIFEST 元数据即可挂载数据，导入速率提升十倍以上。

[M6.3] Rate Limiter 磁盘吞吐限速 (Rate Limiter)

- **吞吐护栏**：后台多线程压实（Compaction）和内存刷盘（Flush）在大规模并发下会吃满物理磁盘的 Write I/O 吞吐，导致前台客户端的读写时延产生数十毫秒的刺突（Latency Spikes）。
- **平滑限速**：通过配置全局 RateLimiter，强行限制后台 Flush 和 Compaction 每秒能写入的最大字节数，将磁盘 I/O 资源平滑分配，确保前台读写操作的时延绝对稳定。

[M6.4] Stats Dump 性能指标审计 (Stats Dump)

- **日志自省**：RocksDB 在后台定时输出或支持主动 API 调用获取详细的运行性能统计大表（Stats Dump）。
- **关键诊断点**：通过分析输出，重点诊断：
 - **Compaction Stats**：各层级读、写放大倍数与 I/O 带宽占用；
 - **Cache Hit Rate**：Index Block 与 Data Block 在 Block Cache 的命中百分比；
 - **Stall Duration**：由于文件堆积导致的写入挂起（Write Stall）总耗时比例，是线上调优的第一数据源。

[M6.5] 表空间损坏与 MANIFEST 修复 (Manifest Recovery)

- **元数据控制**：MANIFEST 文件是整个 LSM 数据库的元数据大本营，记录了当前各层级分别有哪些活跃的 SSTable 文件，以及版本控制链（VersionSet）。
- **修复防线**：若 MANIFEST 文件在物理断电下损坏，会导致数据库无法启动。防线：通过解析数据库目录下的物理 .sst 文件的 Header 元数据，结合残留日志重建 VersionSet 拓扑，重新恢复 MANIFEST 索引大图。

[叁] Zone T RocksDB 性能与诊断实验室 (RocksDB Performance & Diagnostic Labs)

T1 RocksDB 核心调优参数参考对照表

写入与内存参数：
write_buffer_size: MemTable 大小，默认 64MB（高写吞吐可调大至 128MB-256MB）。
max_write_buffer_number: 内存最大 Immutable MemTable 个数，推荐 4 ~ 6。
max_background_jobs: 后台任务并发线程数（包含 Flush 与 Compaction），推荐设为当前系统 CPU 物理核数。
压实限制参数：
level0_file_num_compaction_trigger: 触发 L0 压实的文件数，推荐设为 4。
level0_slowdown_writes_trigger: 触发写限速的 L0 文件数，默认 20。
level0_stop_writes_trigger: 触发写入完全暂停的 L0 文件数，默认 36。

T2 LSM 引擎性能调优金标准

写放大 (Write Amplification) 控制指标：
时序高读写 (FIFO Compaction) → 写放大接近 1-1.5。
通用读写 (Leveled Compaction) → 写放大金标准控制在 10 ~ 25 之间。
内存分配黄金配比：
物理内存分配 → 10% 预留给操作系统内核与 Page Cache | 30% 分配给 RocksDB MemTable (含全局 WriteBufferManager) | 60% 集中分配给 Block Cache。
在 Block Cache 中，必须确保 cache_index_and_filter_blocks=true 以防止元数据被频繁换出。

T3 RocksDB 线上三大瓶颈诊断与调优

- 诊断一：突发 Write Stall 写入断崖式暂停**
 - 现象：客户端写入 QPS 瞬间跌零，后台日志输出 Stalling writes due to L0 files...
 - 调优三步法：
 1. 检查磁盘 I/O 瓶颈，若是 SSD 寿命衰减，开启 RateLimiter 限制后台 Compaction 带宽；
 2. 调大 max_background_jobs（如增加至 8 或 16），分配更多 CPU 核心给后台压实；
 3. 适当调大 level0_slowdown_writes_trigger（如至 30）与 level0_stop_writes_trigger（如至 45）以延迟挂起红线。
- 诊断二：Block Cache 命中率断崖与读时延剧突**
 - 现象：单次点查询 Get 耗时从微秒级飙升至毫秒级，Block Cache 命中率低于 40%。
 - 调优三步法：
 1. 确认已开启 pin_l0_filter_and_index_blocks_in_cache=true，强制锁死元数据；
 2. 确认是否关闭了无用的 Row Cache（若点查询键分布极度稀疏，Row Cache 会抢占大量有效内存，此时应予关闭）；
 3. 增加 Block Cache 分片数（如 num_shard_bits=6 分为 64 片）以降低高并发读下的缓存互斥锁自旋开销。
- 诊断三：SSTable 文件损坏数据库无法启动**
 - 现象：启动报错 Corrupted MANIFEST 或 SST file checksum mismatch
 - 修复三步法：
 1. 使用 ldb 工具命令行检查受损 SSTable: ldb --db=<db_path> check_sst;
 2. 若为 MANIFEST 索引文件逻辑损坏，备份原目录，使用 RepairDB 接口在线扫描物理文件并重建 MANIFEST 大图；
 3. 确认在 DBOptions 中开启了 paranoid_checks=true，及早发现并隔离静默磁盘扇区损坏。